

Table of Contents

TASK 1: THE LONG-RUNNING PROCESS.....	3
Summary.....	3
TASK 2: PROCESS IDENTITY (PID AND PPID).....	3
Summary.....	3
TASK 3: EXIT CODES AND OS FEEDBACK.....	4
Summary.....	4
TASK 4: STANDARD I/O STREAMS.....	5
Summary.....	5
TASK 5: CONDITIONAL EXECUTION AND TERMINATION.....	6
Summary.....	6

Table of Figures

Figure 1: The Long-Running Process.....	3
Figure 2: Process Identity (PID AND PPID).....	3
Figure 3: Exit Codes And Feedback.....	4
Figure 4: Standard I/O Streams.....	5
Figure 5: Conditional Execution And Termination.....	6

TASK 1: THE LONG-RUNNING PROCESS

```
ashim@Ubuntu:~$ cd ~/operating-systems-security-and-networks
ashim@Ubuntu:~/operating-systems-security-and-networks$ mkdir -p week3/task1
ashim@Ubuntu:~/operating-systems-security-and-networks$ nano week3/task1/task1_alive.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ gcc week3/task1/task1_alive.c -o week3/task1/task1
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task1/task1 &
[1] 4675
I am starting...
ashim@Ubuntu:~/operating-systems-security-and-networks$ I am finished.

ashim@Ubuntu:~/operating-systems-security-and-networks$ ps aux | grep task1
ashim      4675  0.0  0.0   2768  1664 pts/0    S   10:06   0:00  ./week3/task1/task1
ashim      4680  0.0  0.0  18000  2620 pts/1    S+  10:06   0:00  grep --color=auto task1
```

Figure 1: The Long-Running Process

Summary

This task demonstrates a long-running Linux process using `sleep()` and a loop. The process is executed in the background and monitored using the `ps` command.

TASK 2: PROCESS IDENTITY (PID AND PPID)

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ nano week3/task2/task2_identity.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ gcc week3/task2/task2_identity.c -o week3/task2/task2
ashim@Ubuntu:~/operating-systems-security-and-networks$ ls -l week3/task2
total 20
-rwxrwxr-x 1 ashim ashim 16136 Sep 22 10:16 task2
-rw-rw-r-- 1 ashim ashim  337 Sep 22 10:15 task2_identity.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task2/task2
My PID is: 5595
My Parent PID is: 4572
Sleeping for 20 seconds...
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task2/task2
My PID is: 5620
My Parent PID is: 4572
Sleeping for 20 seconds...
ashim@Ubuntu:~/operating-systems-security-and-networks$

ashim@Ubuntu:~/operating-systems-security-and-networks$ ps -p 5595 -o pid,ppid,cmd
PID    PPID  CMD
ashim@Ubuntu:~/operating-systems-security-and-networks$ ps -p 5620 -o pid,ppid,cmd
PID    PPID  CMD
5620   4572  ./week3/task2/task2
ashim@Ubuntu:~/operating-systems-security-and-networks$
```

Figure 2: Process Identity (PID AND PPID)

Summary

This task demonstrates how Linux processes identify themselves using PID and PPID. The `getpid()` and `getppid()` functions retrieve the process's own ID and its parent process ID.

TASK 3: EXIT CODES AND OS FEEDBACK

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ mkdir -p week3/task3
ashim@Ubuntu:~/operating-systems-security-and-networks$ nano week3/task3/task3_exit.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ gcc week3/task3/task3_exit.c -o week3/task3/task3
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task3
bash: ./week3/task3: Is a directory
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task3/task3
Enter a number (positive for success, negative for failure): 5
Success
ashim@Ubuntu:~/operating-systems-security-and-networks$ echo $?
0
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task3/task3
Enter a number (positive for success, negative for failure): -5
Failure
ashim@Ubuntu:~/operating-systems-security-and-networks$ echo $?
1
ashim@Ubuntu:~/operating-systems-security-and-networks$
```

Figure 3: Exit Codes And Feedback

Summary

This task demonstrates how a C program communicates its execution status to the operating system using exit codes. return 0 indicates success, while return 1 indicates failure, which can be checked using echo \$?.

TASK 4: STANDARD I/O STREAMS

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ cat week3/task4/task4_input.c
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");

    // Read string from standard input
    scanf("%49s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);

    return 0;
}

ashim@Ubuntu:~/operating-systems-security-and-networks$ mkdir -p week3/task4
ashim@Ubuntu:~/operating-systems-security-and-networks$ nano week3/task4/task4_input.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ gcc week3/task4/task4_input.c -o week3/task4/task4
ashim@Ubuntu:~/operating-systems-security-and-networks$ ls -l week3/task4
total 20
-rwxrwxr-x 1 ashim ashim 16064 Sep 22 10:35 task4
-rw-rw-r-- 1 ashim ashim  224 Sep 22 10:35 task4_input.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task4/task4
Enter your name: Ashim
Hello, Ashim! Welcome to OS Class.
```

Figure 4: Standard I/O Streams

Summary

This task demonstrates standard input and output streams in Linux. The program reads the user's name from stdin using `scanf()` and displays a greeting through stdout using `printf()`.

TASK 5: CONDITIONAL EXECUTION AND TERMINATION

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ cat week3/task5/task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    // Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}

ashim@Ubuntu:~/operating-systems-security-and-networks$ mkdir -p week3/task5
ashim@Ubuntu:~/operating-systems-security-and-networks$ nano week3/task5/task5_control.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ ls -l week3/task5
total 4
-rw-rw-r-- 1 ashim ashim 426 Sep 22 10:42 task5_control.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task5/task5
bash: ./week3/task5/task5: No such file or directory
ashim@Ubuntu:~/operating-systems-security-and-networks$ ^C
ashim@Ubuntu:~/operating-systems-security-and-networks$ gcc week3/task5/task5_control.c -o week3/task5/task5
ashim@Ubuntu:~/operating-systems-security-and-networks$ ls -l week3/task5
total 20
-rwxrwxr-x 1 ashim ashim 16192 Sep 22 10:43 task5
-rw-rw-r-- 1 ashim ashim 426 Sep 22 10:42 task5_control.c
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task5/task5
Current PID: 5901
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
ashim@Ubuntu:~/operating-systems-security-and-networks$ echo $?
0
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/task5/task5
Current PID: 5910
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
ashim@Ubuntu:~/operating-systems-security-and-networks$ echo $?
1
```

Figure 5: Conditional Execution And Termination

Summary

This task demonstrates conditional execution based on user input. The program continues when the user enters 1 and terminates when 0 is entered, returning different exit codes to the OS.